

MODUL AJAR

PEMROGRAMAN INTERNET

Tailwind CSS



DISUSUN OLEH:

Tim Asisten Praktikum Pemrograman Internet
2026/2027

MATERI AJAR

A. Sejarah Tailwind CSS

Sebelum kita melangkah lebih jauh tentang apa itu tailwind? Penting untuk mengetahui sejarah dari Tailwind CSS. Framework ini pertama kali dirilis pada November 2017 oleh Adam Wathan dan timnya. Adam memiliki visi untuk menciptakan sebuah framework CSS yang fleksibel dan dapat disesuaikan dengan kebutuhan proyek, berbeda dari framework tradisional seperti Bootstrap yang lebih kaku.

Tailwind CSS berasal dari kebutuhan Adam untuk menciptakan antarmuka pengguna yang dapat disesuaikan tanpa batasan dari komponen siap pakai. Ia menginginkan sebuah framework yang memungkinkan pengembang untuk membuat desain yang unik tanpa harus menulis CSS yang berulang-ulang. Dengan pendekatan utility-first, Tailwind memungkinkan pengembang untuk menggunakan kelas-kelas kecil yang dapat digabungkan untuk membentuk desain yang kompleks.

Perjalanan Tailwind CSS dimulai dari proyek-proyek pribadi Adam, di mana ia mencoba berbagai pendekatan untuk menyederhanakan proses penulisan CSS. Pada awalnya, Tailwind hanya digunakan dalam lingkup yang terbatas, tetapi karena efektivitas dan fleksibilitasnya, framework ini mulai menarik perhatian komunitas pengembang web.

Pada tahun 2019, popularitas Tailwind CSS meningkat pesat setelah diperkenalkan dalam berbagai konferensi dan podcast pengembangan web. Komunitas pengembang mulai melihat potensi Tailwind dalam mempercepat proses pengembangan dan menjaga konsistensi desain. Banyak pengembang yang sebelumnya skeptis terhadap pendekatan utility-first akhirnya mencoba dan menemukan bahwa Tailwind bisa sangat membantu dalam proyek-proyek mereka.

Seiring berjalannya waktu, Tailwind CSS terus berkembang dengan fitur-fitur baru dan peningkatan yang membuatnya semakin mudah digunakan. Tim pengembang di belakang Tailwind juga aktif dalam merespons umpan balik dari komunitas dan memperbaiki setiap kekurangan yang ada. Hal ini membuat Tailwind CSS menjadi salah satu framework CSS yang paling dicari dan digunakan oleh pengembang web



di seluruh dunia.

B. Apa itu Tailwind?

Apa Itu Tailwind? Tailwind adalah sebuah utility-first CSS framework yang memungkinkan pengembang untuk membangun desain yang kompleks dengan mudah. Tidak seperti framework CSS lainnya yang menyediakan komponen siap pakai, Tailwind memberikan utility classes yang dapat digunakan untuk menyesuaikan tampilan elemen HTML dengan cepat.

Tailwind menawarkan banyak keuntungan yang membuatnya populer di kalangan pengembang web. Pertama, Tailwind sangat fleksibel. Anda bisa mengatur setiap detail dari elemen user interface (UI) tanpa harus menulis CSS custom. Kedua, Tailwind mempercepat proses pengembangan karena Anda tidak perlu keluar dari HTML untuk menulis style. Ketiga, Tailwind memungkinkan penggunaan desain konsisten di seluruh proyek karena semua style diatur melalui utility classes.

C. Kenapa pake Tailwind?

Sebelum menggunakan Tailwind, terdapat beberapa kendala yang cukup mengganggu proses pengembangan komponen UI menggunakan framework CSS konvensional.

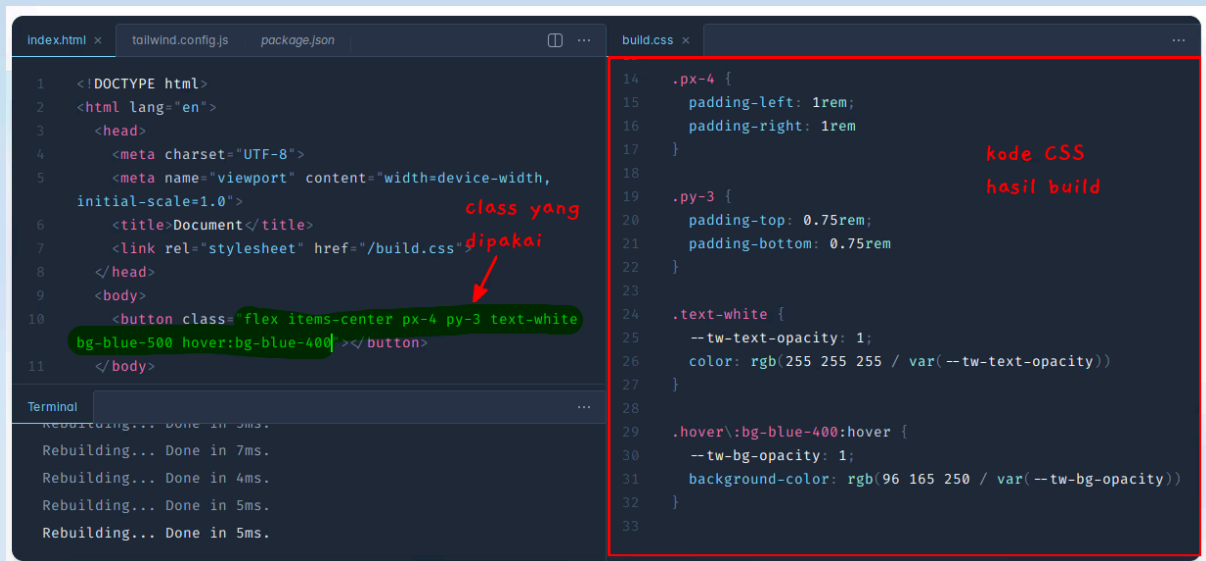
1. Pertama, fleksibilitas dalam melakukan kustomisasi komponen terasa terbatas. Untuk mengubah tampilan suatu komponen, developer umumnya harus melakukan override terhadap class bawaan yang sudah didefinisikan sebelumnya, yang membuat proses styling menjadi kurang efisien.
2. Kedua, kode CSS dari komponen yang sebenarnya tidak digunakan tetap ikut tersimpan dalam hasil build. Hal ini menyebabkan ukuran file CSS akhir menjadi lebih besar dari yang seharusnya. Meskipun sempat dicoba penggunaan PurgeCSS untuk mengatasi masalah tersebut, hasilnya belum sepenuhnya optimal karena beberapa class yang seharusnya tetap ada justru ikut terhapus, sehingga menimbulkan masalah baru pada tampilan.

Dari masalah diatas, Tailwind CSS dipilih sebagai solusi karena mampu



menjawab kedua permasalahan di atas. Tailwind memberikan kebebasan penuh dalam melakukan kustomisasi tampilan tanpa perlu melakukan override, karena setiap komponen dibangun langsung dari kombinasi utility class.

Selain itu, seperti yang terlihat pada contoh di atas, hasil build CSS dari Tailwind hanya akan menyertakan class-class yang benar-benar digunakan di dalam project.



Dalam contoh gambar diatas, hanya class `px-4`, `py-3`, `text-white`, `bg-blue-500`, dan `hover:bg-blue-400` yang muncul pada `build.css`, sesuai dengan class yang digunakan pada elemen `<button>` di `index.html`. Mekanisme ini membuat ukuran file CSS hasil build menjadi jauh lebih ringkas dan efisien, tanpa perlu bergantung pada alat tambahan seperti PurgeCSS.

D. Tools yang Digunakan dalam Tailwind

Untuk memulai menggunakan Tailwind CSS, ada beberapa tools yang perlu Anda ketahui. Tailwind dapat di integrasikan ke dalam proyek Anda melalui beberapa cara, seperti menggunakan CDN atau meng-install melalui NPM.

1. Contoh Syntax Menggunakan Tailwind dari CDN



```

<!doctype html>
<html>
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Contoh Tailwind CDN</title>
    <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
  </head>
  <body>
    <h1 class="text-3xl font-bold underline">
      Hello world!
    </h1>
  </body>
</html>

```

2. Contoh Syntax Project Tailwind dengan NPM

```

// install via npm
npm install tailwindcss @tailwindcss/cli

/* src/input.css */
@import "tailwindcss";

// run build
npx @tailwindcss/cli -i ./src/input.css -o ./src/output.css --watch

<!-- index.html -->
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link href="./output.css" rel="stylesheet">
  </head>
  <body>
    <h1 class="text-3xl font-bold underline">
      Hello world!
    </h1>
  </body>
</html>

```

3. Referensi Dokumentasi

Contoh di atas menggunakan pendekatan Tailwind CLI, yaitu cara paling sederhana untuk memakai Tailwind di project HTML biasa tanpa bundler seperti Vite atau Webpack. Tailwind CLI cukup membaca file CSS sumber (input.css), memindai class yang dipakai di HTML/JS project, lalu membuat file CSS hasil compile (output.css) yang berisi hanya utility class yang benar-benar dipakai.



Untuk dokumentasi resmi & lengkap mengenai instalasi via CLI, kamu bisa cek di:

<https://tailwindcss.com/docs/installation/tailwind-cli>

Tutorial tailwind dari geeksforgeeks:

<https://www.geeksforgeeks.org/css/tailwind-css/>

E. Anatomy Utility Class

Setiap utility class Tailwind mewakili satu deklarasi CSS. Nama classnya mengikuti pola properti diikuti nilainya, sehingga begitu terbiasa dengan polanya, sebagian besar class dapat ditebak tanpa membuka dokumentasi.

Utility Class	Setara dengan CSS	Pola Penamaan
p-4	padding: 1rem;	p = padding, angka = kelipatan 0.25rem
mt-2	margin-top: 0.5rem;	m = margin, t = top
text-xl	font-size: 1.25rem;	text- diikuti nama ukuran
bg-blue-500	background-color: #3b82f6;	bg- diikuti warna dan tingkat
w-full	width: 100%;	w = width, full = 100%
rounded-lg	border-radius: 0.5rem;	rounded- diikuti ukuran sudut

Dua skala yang paling sering dipakai adalah skala jarak dan skala warna.

- Skala jarak memakai kelipatan 0.25rem atau 4px. Jadi p-1 sama dengan 4px, p-2 sama dengan 8px, dan p-4 sama dengan 16px.
- Skala warna memakai tingkat 50 sampai 950. Angka kecil berarti makin terang, angka besar berarti makin gelap, misalnya bg-slate-50 sampai bg-slate-900.

F. Utility untuk Layout

Utility layout menentukan bagaimana elemen ditampilkan dan disusun. Tailwind tidak mengubah konsep CSS yang sudah dipelajari sebelumnya, hanya meningkatnya menjadi class.



Utility Class	Setara dengan CSS	Kegunaan
block, inline, hidden	display: block / inline / none	Mengatur cara elemen tampil, hidden menyembunyikan elemen sepenuhnya.
flex	display: flex	Menjadikan elemen sebagai flex container.
flex-col	flex-direction: column	Mengubah arah sumbu utama jadi vertikal.
justify-between	justify-content: space-between	Mengatur perataan item di sumbu utama.
items-center	align-items: center	Mengatur perataan item di sumbu silang.
grid, grid-cols-3	display: grid; grid-template-columns: repeat(3, 1fr)	Membuat grid dengan tiga kolom sama lebar.
gap-4	gap: 1rem	Jarak antar item pada flex maupun grid.
relative, absolute, fixed, sticky	position: ...	Menentukan metode penempatan elemen, sama seperti CSS biasa.

HTML

```
<nav class="flex items-center justify-between p-4">
  <span class="font-bold">KopiKita</span>
  <ul class="flex gap-6">
    <li><a href="#">Beranda</a></li>
    <li><a href="#">Menu</a></li>
  </ul>
</nav>
```

G. Utility untuk Spacing dan Sizing

Utility Class	Setara dengan CSS	Kegunaan
p-4, px-4, py-2, pt-3	padding	p semua sisi, px kiri-kanan, py atas-bawah, pt hanya atas.
m-4, mx-auto, mb-6	margin	mx-auto dipakai untuk menengahkan elemen yang punya lebar tetap.



Utility Class	Setara dengan CSS	Kegunaan
w-full, w-1/2, w-64	width	Lebar penuh, setengah, atau ukuran tetap.
h-screen	height: 100vh	Tinggi elemen setinggi satu layar penuh.
max-w-4xl	max-width	Membatasi lebar maksimum agar teks tetap nyaman dibaca.
space-y-4	margin antar anak	Memberi jarak vertikal antar elemen anak tanpa menulis margin satu per satu.

H. Utility untuk Tipografi dan Warna

Utility Class	Setara dengan CSS	Kegunaan
text-sm sampai text-6xl	font-size	Ukuran huruf dari kecil ke besar.
font-normal, font-bold	font-weight	Ketebalan huruf.
italic	font-style: italic	Membuat teks miring.
leading-relaxed	line-height	Jarak antar baris teks.
tracking-wide	letter-spacing	Jarak antar huruf.
text-center, text-justify	text-align	Perataan teks.
underline, no-underline	text-decoration	Menambah atau menghapus garis bawah.
text-slate-700	color	Warna teks.
bg-amber-100	background-color	Warna latar belakang.
border-slate-300	border-color	Warna garis tepi.

HTML

```
<h1 class="text-4xl font-bold tracking-tight text-slate-800">
  Festival Kuliner Kampus
</h1>
<p class="mt-2 leading-relaxed text-slate-600">
  Hadir lagi tahun ini dengan lebih banyak tenant.
</p>
```



I. Utility untuk Border, Radius, dan Shadow

Utility Class	Setara dengan CSS	Kegunaan
border, border-2	border-width	Ketebalan garis tepi.
rounded-md, rounded-xl, rounded-full	border-radius	Kelengkungan sudut, rounded-full membentuk lingkaran penuh.
shadow-sm, shadow-md, shadow-lg	box-shadow	Bayangan dari yang paling tipis sampai paling tebal.
overflow-hidden	overflow: hidden	Memotong isi yang keluar dari kotak, berguna pada kartu bergambar.

HTML

```
<article class="rounded-xl border border-slate-200 p-5 shadow-md">
  <h3 class="text-lg font-bold">Nasi Goreng Kampus</h3>
  <p class="text-slate-600">Rp15.000</p>
</article>
```

J. Responsive Design dengan Breakpoint Prefix

Tailwind menerapkan pendekatan mobile-first. Class tanpa prefix berlaku untuk semua ukuran layar, sedangkan class dengan prefix hanya aktif mulai dari lebar layar tertentu ke atas. Prefix ditulis di depan utility dan dipisahkan tanda titik dua.

Prefix	Aktif Mulai Lebar	Contoh Pemakaian
sm:	640px	sm:text-lg
md:	768px	md:grid-cols-2
lg:	1024px	lg:grid-cols-3
xl:	1280px	xl:max-w-6xl
2xl:	1536px	2xl:px-20

HTML

```
<div class="grid grid-cols-1 gap-4 md:grid-cols-2 lg:grid-cols-3">
  <!-- satu kolom di ponsel, dua di tablet, tiga di desktop -->
</div>
```

Perhatikan urutan penulisannya: tulis dulu tampilan untuk layar kecil, baru



tambahkan prefix untuk layar yang lebih besar. Pola inilah yang membuat Tailwind terasa mobile-first.

K. State Variant

Selain breakpoint, prefix juga dipakai untuk menandai kondisi elemen. Cara penulisannya sama, yaitu nama kondisi diikuti tanda titik dua lalu utility yang ingin diterapkan.

Variant	Kondisi	Contoh Pemakaian
hover :	Kursor berada di atas elemen.	hover:bg-blue-600
focus :	Elemen sedang aktif atau dipilih, umumnya pada input.	focus:ring-2
active :	Elemen sedang ditekan.	active:scale-95
disabled :	Elemen dalam keadaan nonaktif.	disabled:opacity-50
group-hover :	Kursor berada di atas elemen induk yang diberi class group.	group-hover:text-white

HTML

```
<button class="rounded-lg bg-blue-500 px-4 py-2 text-white
              hover:bg-blue-600 focus:ring-2 active:scale-95">
  Kirim
</button>
```

L. Dark Mode

Tailwind menyediakan variant dark: untuk menuliskan tampilan alternatif pada mode gelap. Pada pemakaian lewat CDN, variant ini secara bawaan mengikuti preferensi tema sistem operasi pengguna, sehingga tidak perlu pengaturan tambahan.

HTML

```
<section class="bg-white text-slate-800 dark:bg-slate-900 dark:text-slate-100">
  <h2 class="text-2xl font-bold">Dies Natalis UPI</h2>
</section>
```

Pola penulisannya selalu berpasangan: satu class untuk mode terang, satu class lagi dengan prefix dark: untuk mode gelap. Jika ingin mode gelap dikendalikan tombol, bukan oleh sistem, Tailwind perlu diberi tahu bahwa penanda mode gelap adalah sebuah class pada elemen induk.



HTML

```
<style type="text/tailwindcss">
  @custom-variant dark (&:where(.dark, .dark *));
</style>

<html class="dark"> ... </html>  <!-- class dark ditambah lewat tombol -->
```

M. Kustomisasi Tema

Warna dan font bawaan Tailwind dapat ditambah sesuai identitas proyek. Pada pemakaian lewat CDN, kustomisasi ditulis di dalam tag style dengan tipe `text/tailwindcss` memakai aturan `@theme`. Setiap variabel yang didefinisikan akan otomatis tersedia sebagai utility class.

HTML

```
<script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>

<style type="text/tailwindcss">
  @theme {
    --color-brand: #1f3a8a;
    --color-brand-soft: #dbe6ff;
    --font-display: "Poppins", sans-serif;
  }
</style>

<h1 class="font-display text-brand">Halo</h1>
<div class="bg-brand-soft p-6">Kotak dengan warna kustom</div>
```

Perhatikan hubungan namanya: variabel `--color-brand` melahirkan utility `bg-brand`, `text-brand`, dan `border-brand`, sedangkan `--font-display` melahirkan utility `font-display`.

Catatan tentang instalasi lewat npm dan Vite

Pada praktikum ini kita cukup memakai CDN karena tidak memerlukan pemasangan apa pun dan hasilnya langsung terlihat di browser. Perlu diketahui bahwa proyek nyata umumnya memasang Tailwind melalui npm dengan bantuan bundler seperti Vite. Alurnya kurang lebih seperti berikut.

Shell

```
npm create vite@latest nama-proyek
cd nama-proyek
npm install tailwindcss @tailwindcss/vite
npm run dev
```



Keuntungannya, berkas CSS hasil build hanya berisi class yang benar-benar dipakai sehingga ukurannya kecil, tersedia fitur seperti autocomplete dan hot reload, dan tidak ada ketergantungan pada koneksi internet saat halaman dibuka. Kekurangannya, perlu Node.js dan proses build, yang belum diperlukan untuk latihan sekelas praktikum ini. Materi ini cukup dipahami sebagai pengetahuan, tidak dipraktikkan pada pertemuan ini.

N. Komponen yang Dapat Digunakan Kembali

Ketika sebuah kombinasi class dipakai berulang, misalnya pada kartu produk yang muncul enam kali, ada dua cara menanganinya.

1. Mengulang kombinasi utility class

Kombinasi class ditulis ulang pada setiap elemen. Cara ini paling jujur, karena tampilan sebuah elemen dapat dibaca langsung dari HTML tanpa membuka berkas lain. Kekurangannya, saat desain berubah, seluruh salinan harus diubah satu per satu.

2. Menyatukan class dengan @apply

Kombinasi class dikemas menjadi satu nama class baru. Perubahan desain cukup dilakukan di satu tempat, tetapi tampilan elemen tidak lagi terbaca langsung dari HTML, sehingga pemakaiannya perlu dibatasi pada pola yang benar-benar sering berulang.

```
HTML
<style type="text/tailwindcss">
  .kartu {
    @apply rounded-xl border border-slate-200 p-5 shadow-md;
  }
</style>

<article class="kartu">Produk A</article>
<article class="kartu">Produk B</article>
```

Pada proyek yang memakai framework seperti React atau Laravel Blade, cara ketiga yang paling umum adalah menjadikannya komponen, sehingga kombinasi class cukup ditulis sekali di dalam komponen tersebut.



O. Perbandingan dengan CSS Konvensional

Aspek	CSS Konvensional	Tailwind CSS
Tempat menulis gaya	Berkas .css terpisah dengan selector sendiri	Langsung pada atribut class di HTML
Penamaan	Perlu memikirkan nama class yang bermakna	Tidak perlu, nama class sudah baku
Konsistensi	Bergantung disiplin penulis	Terjaga karena memakai skala yang sama
Ukuran berkas	Semua aturan ikut terbawa walau tidak dipakai	Hanya class yang dipakai yang ikut pada hasil build
Keterbacaan HTML	Ringkas, gaya tidak terlihat di HTML	Atribut class menjadi panjang
Kurva belajar	Perlu hafal properti CSS	Perlu hafal nama utility, tetapi polanya konsisten



HANDS ON

Langkah 1 : Buat file HTML kosong

Buat file baru bernama `index.html`.

Langkah 2 : Tulis struktur dasar HTML

Isi kerangka dasar HTML:

```
HTML
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

Langkah 3 : Tempelkan script Tailwind di dalam <head>

Satu baris <script> ini akan mengaktifkan seluruh utility class Tailwind langsung di browser, tanpa build tool apa pun.

```
HTML
<script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4">
</script>
```

Langkah 4 : Coba pakai utility class di <body>

Sekarang kamu bisa langsung menulis class Tailwind pada elemen HTML apa pun. Tambahkan kode ini di dalam <body>:

```
HTML
<h1 class="text-3xl font-bold underline">
  Hello world!
</h1>
```

Langkah 5 : Coba Warnai teks & latar (color)

Pola nama: **text-{warna}-{tingkat}** untuk warna teks, **bg-{warna}-{tingkat}** untuk latar belakang. Tingkat dari 50 (paling terang) sampai 900 (paling gelap).



Contoh penerapan :

```
HTML
<p class="bg-yellow-100 text-blue-600">
  Teks Biru
</p>
```

Langkah 6 : Ukuran & ketebalan teks (typography)

Ukuran teks dari **text-xs** (paling kecil) sampai **text-6xl** (sangat besar).
font-bold membuat teks tebal.

Contoh penerapan :

```
HTML
<h2 class="text-xl font-bold">
  Judul Tebal
</h2>
```

Langkah 7 : Jarak dalam & luar (padding & margin)

p = padding (jarak di dalam elemen), **m = margin** (jarak di luar elemen).
Angka mewakili kelipatan 0.25rem, jadi p-4 = 1rem.

Contoh penerapan :

```
HTML
<div class="bg-blue-100 p-4 rounded-md">
  Kotak dengan p-4
</div>
```

Langkah 8 : Sudut & bayangan (border-radius & shadow)

Pola nama: **rounded-{ukuran}** untuk sudut (dari sm sampai full untuk lingkaran penuh), dan **shadow-{ukuran}** untuk bayangan (dari sm yang tipis sampai xl yang tebal).

Contoh penerapan :

```
HTML
<div class="bg-white p-4 rounded-xl shadow-md">
  Kartu dengan shadow
</div>
```

Langkah 9 : Tata letak (flexbox)



Pola nama: **flex** mengaktifkan mode flexbox pada elemen. **justify-{posisi}** mengatur posisi horizontal (center, between, dst), **items-{posisi}** mengatur posisi vertikal, dan **gap-{angka}** mengatur jarak antar elemen di dalamnya.

Contoh penerapan :

HTML

```
<div class="flex gap-2">
  <div class="w-8 h-8 bg-teal-600 rounded-md"></div>
  <div class="w-8 h-8 bg-teal-300 rounded-md"></div>
</div>
```

Langkah 10 : Susun kartu dengan grid

grid-cols-{jumlah} membagi area menjadi beberapa kolom sama lebar, **gap** mengatur jarak antar kartu.

Contoh penerapan :

HTML

```
<div class="grid grid-cols-3 gap-4">
  <div class="bg-white p-4 rounded-xl shadow-md">Kartu 1</div>
  <div class="bg-white p-4 rounded-xl shadow-md">Kartu 2</div>
  <div class="bg-white p-4 rounded-xl shadow-md">Kartu 3</div>
</div>
```

Langkah 11 : Reaksi saat disentuh kursor (hover & focus)

Tambahkan prefix **hover:** atau **focus:** di depan utility yang ingin diterapkan hanya pada kondisi tersebut.

Contoh penerapan :

HTML

```
<button class="bg-blue-500 text-white px-4 py-2 rounded-lg
              hover:bg-blue-600 focus:ring-2">
  Daftar Sekarang
</button>
```

Langkah 12 : Membuat tampilan responsif

Tulis dulu tampilan untuk layar kecil, lalu tambahkan prefix **md:** atau **lg:** untuk layar yang lebih besar. Coba perkecil jendela browser untuk melihat perubahannya.

Contoh penerapan :



HTML

```
<div class="grid grid-cols-1 gap-4 md:grid-cols-2 lg:grid-cols-3">
  <div class="bg-white p-4 rounded-xl shadow-md">Kartu</div>
</div>
```

Langkah 13 : Mode gelap

Pasangkan class mode terang dengan class bertanda dark:. Ubah tema sistem operasi menjadi gelap untuk mencobanya.

Contoh penerapan :

HTML

```
<section class="bg-white text-slate-800 p-6
               dark:bg-slate-900 dark:text-slate-100">
  Halo, coba ganti tema perangkatmu.
</section>
```

Langkah 14 : Warna dan font sendiri

Tambahkan blok @theme di dalam tag style bertipe text/tailwindcss, lalu pakai namanya sebagai utility class biasa.

Contoh penerapan :

HTML

```
<style type="text/tailwindcss">
  @theme {
    --color-kampus: #1f3a8a;
  }
</style>

<div class="bg-kampus text-white p-6 rounded-xl">Warna kampus</div>
```

Selamat Mengerjakan!!

